Architectural and Algorithmic Analysis of a Web-Based Rubik's Cube Solver: 3D Kinematics, State Management, and Kociemba's Two-Phase Algorithm

Technical Documentation

May 24, 2026

Abstract

This document provides a comprehensive technical analysis of a web-based Rubik's Cube solver application. The system integrates a WebGL-powered 3D rendering pipeline using Three.js for kinematic visualization and user interaction, alongside a robust mathematical model for state representation. The core computational engine relies on Herbert Kociemba's Two-Phase Algorithm to compute near-optimal solutions in real-time. We detail the state-space reduction techniques, the construction of move and pruning tables via Iterative Deepening A* (IDA*), bitwise memory optimization strategies, and the constrained application of the Fisher-Yates shuffle algorithm for generating mathematically valid random states.

Contents

1 Introduction

The Rubik's Cube possesses a state space of approximately 4.33×10^{19} valid configurations. Solving this puzzle optimally (finding God's Number, which is 20 in the half-turn metric) is computationally expensive for real-time web applications. To bridge the gap between optimal solving and real-time performance, this application implements Herbert Kociemba's Two-Phase Algorithm, which reduces the problem into two manageable subgroup searches. The application is built on a Node.js/Express backend serving a client-side HTML5/JavaScript frontend, utilizing WebGL for 3D rendering and raycasting for gesture-based manipulation.

2 System Architecture and Workflow

The application follows a Model-View-Controller (MVC) paradigm, heavily decoupling the mathematical state (Model) from the 3D representation (View).

2.1 3D Rendering and Interaction Pipeline

The visual representation is managed by `CubeRenderer` using the Three.js library. The cube is constructed from 26 individual `Mesh` objects (cubelets) grouped under a `THREE.Group`.

- **Raycasting and Gestures:** User interaction relies on `THREE.Raycaster`. Upon a `pointerdown` event, the ray intersects the cubelet mesh to determine the face normal $\vec{n}$.
- **Drag Vector Projection:** As the user drags, a drag vector $\vec{v}$ is computed on the plane coplanar to $\vec{n}$. The rotation axis $\vec{r}$ is determined via the cross product $\vec{r} = \vec{n} \times \vec{v}$.
- **Move Mapping:** The axis $\vec{r}$ and the spatial coordinates of the manipulated cubelet are mapped to standard Singmaster notation (e.g., $U, R', F2$) via a predefined hash map (`AXIS_TO_MOVE`).

2.2 State Synchronization

The mathematical model (`cube.js`) represents the cube state independently of the 3D scene. When a move is executed:

1. The algebraic state is updated via permutation and orientation multiplication.
2. A 54-character string is generated representing the color of each facelet.
3. The `CubeRenderer` parses this string and updates the `MeshPhongMaterial` color properties of the corresponding 3D cubelets.
4. The 2D DOM unfolded input grid is updated synchronously.

3 Mathematical State Representation

The cube state is modeled using the disjoint cycle notation of permutations and orientation vectors, avoiding the overhead of tracking 54 individual facelets.

3.1 Cubie Model

The state is defined by four arrays:

- **Corner Permutation (`cp`):** An array of 8 integers representing the physical locations of the corner cubies.
- **Corner Orientation (`co`):** An array of 8 integers $\in \{0, 1, 2\}$ representing the twist of each corner.
- **Edge Permutation (`ep`):** An array of 12 integers for edge locations.
- **Edge Orientation (`eo`):** An array of 12 integers $\in \{0, 1\}$ representing edge flips.

3.2 Constrained Randomization (Fisher-Yates)

To generate a scrambled cube, the application utilizes the Fisher-Yates shuffle algorithm [2]. However, a standard shuffle can generate $8! \times 12! \times 3^8 \times 2^{12}$ states, many of which are physically impossible to reach by legal face turns. The algorithm is constrained by the fundamental laws of the Rubik's Cube:

1. **Parity Constraint:** The sign of the corner permutation must equal the sign of the edge permutation: $\text{sgn}(\texttt{cp}) = \text{sgn}(\texttt{ep})$.
2. **Twist Constraint:** The sum of corner orientations must be divisible by 3: $\sum \texttt{co}_i \equiv 0 \pmod 3$.
3. **Flip Constraint:** The sum of edge orientations must be divisible by 2: $\sum \texttt{eo}_i \equiv 0 \pmod 2$.

The implementation applies the $O(N)$ Fisher-Yates shuffle and subsequently rejects and resamples the arrays until all three invariants are satisfied.

4 Herbert Kociemba's Two-Phase Algorithm

The core solver (`solve.js`) implements Kociemba's algorithm, which seeks a near-optimal solution (typically 20-22 moves) in milliseconds by dividing the search space into two distinct phases [1].

4.1 Phase 1: Reduction to Subgroup G_1

The goal of Phase 1 is to move the cube from the arbitrary group $G = \langle U, R, F, D, L, B \rangle$ into the subgroup $G_1 = \langle U, D, R2, L2, F2, B2 \rangle$. In G_1, the orientation of all edges and corners is fixed, and the middle-layer (slice) edges are confined to the middle slice.

The state in Phase 1 is uniquely identified by three coordinates:

1. **Twist:** The orientation of the 8 corners. Since the total twist is conserved, the 8th corner is dependent on the first 7. State space: $3^7 = 2187$.
2. **Flip:** The orientation of the 12 edges. The 12th is dependent. State space: $2^{11} = 2048$.
3. **Slice:** The positions of the 4 middle-layer edges (FR, FL, BL, BR) among the 12 available edge slots. State space: $\binom{12}{4} = 495$.

4.1.1 Move Tables and Pruning Tables

To navigate this space, the application precomputes *Move Tables* (transition functions) and *Pruning Tables* (lower-bound heuristic estimates).

- **Move Tables:** A 2D array where $M[c][m]$ yields the new coordinate c' after applying move m.
- **Pruning Tables:** Stores the minimum number of moves required to reach the goal state (coordinate 0) from any coordinate c.

4.1.2 Bitwise Memory Optimization

To minimize memory footprint and maximize cache locality, the Pruning Tables are compressed. Since the maximum depth in Phase 1 is rarely above 12, a 4-bit integer is sufficient to store the depth. The application packs eight 4-bit depth values into a single 32-bit signed integer.

Algorithm 1 Bitwise Pruning Table Access

```
function GetPruningValue(table, index)
    pos ← index (mod 8)
    slot ← ⌊index/8⌋                          ▷ Bitwise: index » 3
    shift ← pos × 4                            ▷ Bitwise: pos « 2
    return (table[slot] AND (15 ≪ shift)) ≫ shift
end function
```

4.2 Phase 2: Solving within G_1

Once the cube is in G_1, only the moves $\{U, D, R2, L2, F2, B2\}$ are permitted. The state is tracked using:

- **Corner Permutation:** Positions of 6 corners (URF to DLF). Size: $8!/2! = 20160$.
- **Edge Permutation:** Positions of 6 edges (UR to DF). Size: 20160.
- **Slice Permutation:** Arrangement of the 4 slice edges. Size: $4! = 24$.
- **Parity:** Corner/Edge parity. Size: 2.

Phase 2 utilizes similar Move and Pruning tables, searching for the identity state using the restricted move set.

4.3 Search Strategy: Iterative Deepening A* (IDA*)

Both phases employ IDA* to find the shortest path. Let $h(n)$ be the heuristic from the Pruning Table, and $g(n)$ be the current depth.

Algorithm 2 IDA* Search (Phase 1)

```
procedure Phase1Search(state)
    for depth = 1 to MAX_DEPTH do
        if Search(state, depth, 0) then
            return Solution
        end if
    end for
end procedure
procedure Search(state, max_d, current_d)
    h ← max(Pruning_{twist, slice}, Pruning_{flip, slice})
    if current_d + h > max_d then
        return false                                          ▷ Prune branch
    end if
    if h == 0 then
        Phase2Search(state)
    end if
    for move in ValidMoves(state.last_move) do
        next_state ← ApplyMove(state, move)
        if Search(next_state, max_d, current_d + 1) then
            return true
        end if
    end for
    return false
end procedure
```

To prevent redundant sequences, the algorithm enforces a move-ordering constraint (e.g., avoiding U followed by U', and enforcing a strict order for parallel moves like U then D, never D then U).

5 Conclusion

The application demonstrates a highly optimized pipeline for combinatorial puzzle solving in a web environment. By decoupling the 3D kinematics from the algebraic state, utilizing memory-efficient bitwise pruning tables, and leveraging Kociemba's subgroup reduction strategy, the solver achieves real-time performance. The integration of constrained Fisher-Yates shuffling ensures that all generated user inputs and random scrambles remain within the valid mathematical manifold of the Rubik's Cube.

References

[1] Herbert Kociemba. (2021). *Group Theory and the Rubik's Cube*. http://kociemba.org/cube.htm

[2] Fisher, R. A., & Yates, F. (1938). *Statistical tables for biological, agricultural and medical research*. London: Oliver and Boyd. (As implemented via bitwise optimized JavaScript shuffling, StackOverflow, 2010).

[3] Three.js Contributors. (2023). *Three.js: JavaScript 3D Library*. https://threejs.org/docs/

[4] Singmaster, D. (1981). *Notes on Rubik's "Magic Cube"*. Penguin Books.